
SCOPE: Evolving Symbolic World for Planning in Open-Ended Environments

Yundaichuan Zhan^{*1} Minghe Gao^{*1} Zhongqi Yue² Wendong Bu¹ Wenqiao Zhang¹ Guoming Wang¹
Jisheng Dang³ Juncheng Li¹ Siliang Tang¹ Yueting Zhuang¹

Abstract

Recent works have explored integrating Vision-Language Models (VLMs) with classical planners that rely on symbolic representations of planning problem to generate long-horizon plans for complex embodied tasks. However, in open-ended environments, these symbolic representations obtained from perception are often incomplete, leading to suboptimal performance. To address this, we introduce SCOPE, a self-adaptive symbolic planning framework that supports refining action plans and evolving the symbolic world—the symbolic representations of open-ended environments. SCOPE comprises two synergistic modules: a Symbolic Execution Simulator (SESIM) that conducts symbolic validation and real execution of action plans, leveraging the feedback to refine the plans and evolve the symbolic world; and a Self-Adaptive Symbolic Memory (SASMem) that further distills feedback into evolving symbolic knowledge to enhance long-horizon planning and modeling of the symbolic world. Experiments in open-ended environments show that SCOPE significantly improves the completeness of the symbolic world, the success rate of plans under environment perturbations, and cross-task grounding and adaptability across diverse embodied scenarios.

1. Introduction

Vision-Language Models (VLMs), powered by large-scale multimodal pretraining (Du et al., 2022; Driess et al., 2023; Alayrac et al., 2022; Achiam et al., 2023; Hurst et al., 2024; Yang et al., 2025), excel at locating objects (Chen et al., 2024; Ao et al., 2025; Zeng et al., 2022), recognizing spatial

relations (Lu et al., 2016; Cheng et al., 2024; Fei et al., 2024), and interpreting goal-relevant cues from images and text (Yang et al., 2023). These capabilities make VLMs strong candidates for serving as the agent to plan actions for embodied tasks.

However, open-ended environments are characterized by diverse and evolving task-relevant objects/relations/states that cannot be exhaustively pre-specified. In such embodied environments that involve intricate object properties and long sequences of interdependent actions, VLMs without task-specific training often generate ungrounded or hallucinated actions that violate causal logic (Wu et al., 2024; Huang et al., 2022b; Ahn et al., 2022; Liang et al., 2022; Wong et al., 2023; Huang et al., 2022a). These missteps tend to accumulate over time, ultimately undermining the reliability of planning.

To address this, recent approaches incorporate classical symbolic planners alongside large models by generating symbolic representations of the planning problem (Fung et al., 2025; Xiong et al., 2025; Dwivedi et al., 2023; Zhang et al., 2023; Silver et al., 2022), in order to mitigate hallucinated actions and causal violations. Symbolic representations could be expressed using the Planning Domain Definition Language (PDDL) (Aeronautiques et al., 1998; Zhi-Xuan, 2022), which separates a planning problem into a domain file that defines predicates, action schemas and transition rules, and a problem file that specifies objects, initial states, and goal conditions.

Nonetheless, as illustrated in Figure 1, these attempts to integrate large models with classical planners have revealed two challenges in modeling of symbolic representations and long-horizon planning: **(1) Incomplete and Static Symbolic World:** At the perception stage, existing methods (Liu et al., 2023; Migimatsu & Bohg, 2022) construct the symbolic world by using a VLM to ground observations and instructions into symbolic representations (objects and their symbolic states/relations in the PDDL problem). Yet in open-ended environments, many critical states and affordances are action-dependent and only revealed through interaction, making one-shot symbolic grounding inevitably incomplete. **(2) Non-adaptive Symbolic Knowledge:** While prior methods may incorporate feedback or it-

^{*}Equal contribution ¹Zhejiang University, Hangzhou, China

²Chalmers University of Technology, Gothenburg, Sweden

³Lanzhou University, Lanzhou, China. Correspondence to: Juncheng Li <junchengli@zju.edu.cn>.

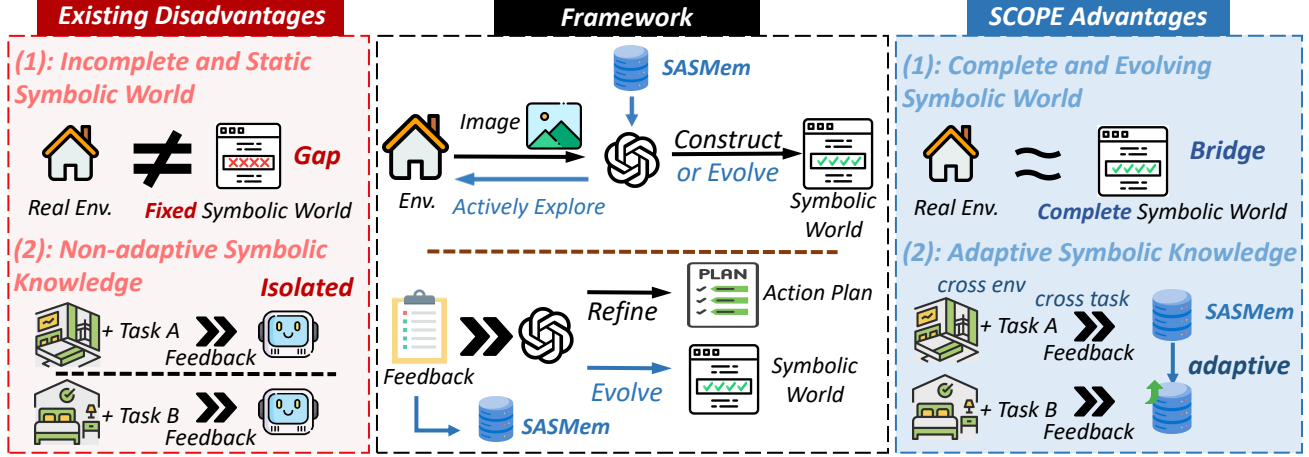


Figure 1. Comparison of existing method and SCOPE. In the middle illustration, **black arrows** denote the pipeline used by existing methods; **blue arrows** highlight SCOPE extensions.

erative refinement, the resulting experience is often recorded as task-specific traces and is not often distilled into symbolic-friendly knowledge that is aligned with symbolic planning. As a consequence, the stored information has limited density and transferability, providing weaker cross-task and cross-scene support for symbolic planning and symbolic world refinement (Sarch et al., 2024; Zhou et al., 2024).

Inspired by the paradigm of reasoning with iterative symbolic correction (Zhou et al., 2024; Mao et al., 2019; Zhu et al., 2021), we propose **SCOPE**, a self-adaptive symbolic planning framework that constructs the symbolic world from perception and evolves the symbolic world and refines action plans through feedback from symbolic validation and real execution. Specifically, as shown in Figure 2, **SCOPE** evolves the symbolic world as the agent actively explores the open-ended environment. Through the Symbolic Execution Simulator (**SESim**), it could validate the logical consistency between the plan and the symbolic world, and generate the feedback from both symbolic validation and real execution to achieve grounded reasoning and drive exploration to construct a **complete and evolving symbolic world**. In addition, feedback is distilled into a Self-Adaptive Symbolic Memory (**SASMem**), enabling **adaptive symbolic knowledge** that enhances grounding across tasks and adaptability across diverse embodied environments.

Importantly, the two modules in **SCOPE** form a tightly coupled closed-loop framework that continually improves over time. The SASMem distills feedback into symbolic knowledge that enhances VLM’s capability to generate more accurate symbolic worlds and action plans in future tasks. In turn, the SESim performs symbolic validation and real execution. Then, it provides feedback not only for plan refining but also for updating SASMem.

Extensive experiments validate that evolving symbolic

world representations and abstracting reusable symbolic knowledge across tasks and across diverse embodied environments, enable strong continual adaptation, its step success rate improves by 14.4% throughout a continual run. Furthermore, in open-ended environments, this self-adaptive behavior requires fewer symbolic refinements and exhibits improved robustness compared to the NESYC baseline (Choi et al., 2025), achieving an average increase of 11.3% in task success rate across all evaluated open-ended tasks. In general, our contributions are as follows:

- **SCOPE** establishes a self-adaptive symbolic planning framework that evolves the symbolic world over time, providing a foundation for grounded planning within symbolic representations.
- Within **SCOPE**, the SESim integrates symbolic validation and real execution to narrow the representation gap between the symbolic world and the open-ended environment, and support more grounded action plans.
- **SCOPE** further incorporates the SASMem, which distills reusable feedback into action reasoning and world modeling knowledge aligned with symbolic planning, enabling cross-task grounding and adaptability across diverse embodied scenarios.

2. Related Works

2.1. Planning with Large Pretrained Models and Symbolic Feedback

Recent efforts have investigated enhancing the planning capabilities of Large Pretrained Models through prompts and symbolic feedback (Akakzia et al., 2020; Jiang et al., 2019a; Mirchandani et al., 2021; Silver et al., 2023; Srinivas et al., 2018; Nair & Finn, 2019; Choi et al., 2025). For example,

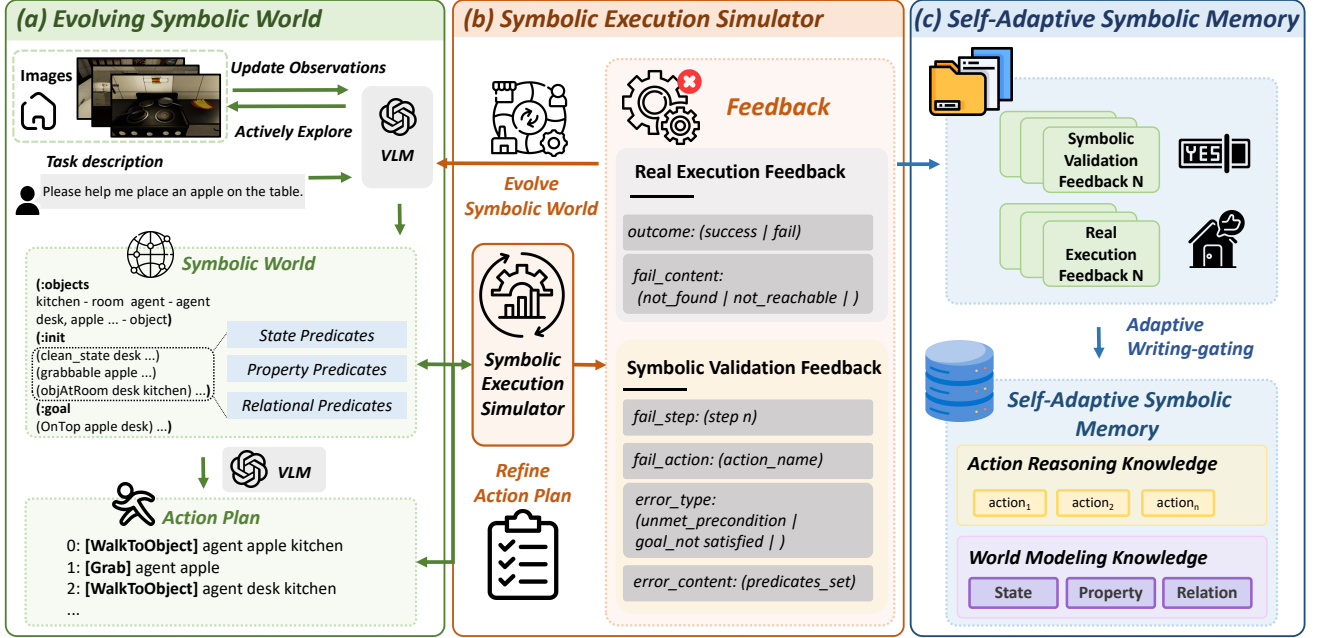


Figure 2. The overview of our framework. (a) Evolving Symbolic World: The agent actively explores the environment to refine the symbolic world, represented in PDDL problem file. Based on this symbolic world, the VLM generates a symbolic action plan for the embodied task. (b) Symbolic Execution Simulator: The generated plan is validated within the symbolic world using the PDDL validator and executed in the environment. SESim integrates feedback from symbolic validation and real execution to evolve the symbolic world and refine the action plan, narrowing the gap between symbolic and real environments. (c) Self-Adaptive Symbolic Memory: Feedback is distilled into abstract knowledge of action planning and world modeling. This knowledge is retrieved on demand to provide targeted hints for plan/world refinement.

ProgPrompt (Singh et al., 2023) proposes a programmatic prompt format that embeds executable examples and action templates in the LLM input. ISR-LLM (Zhou et al., 2024) further improves symbolic consistency through an iterative self-refinement process: Employ a classical validator to refine the LLM-generated plan over multiple rounds. This symbolic feedback loop increases plan feasibility while maintaining generalization from language.

However, these methods mainly focus on refining action plans for a single task and do not accumulate symbolic feedback over time. In contrast, **SCOPE** distills feedback into reusable symbolic knowledge that improves both symbolic world modeling and action reasoning across tasks and environments, enabling more robust and adaptive planning.

2.2. Symbolic Modeling for Embodied Planning

PDDL-based symbolic abstraction employs LLMs to translate natural language into symbolic domains and problem definitions, which are then solved using classical planners (Liu et al., 2023; Hu et al., 2025; Zhang et al., 2025; Jiang et al., 2019b). However, symbolic representations in prior works are typically static and incomplete. They are often handcrafted by experts, extracted from simulators using rule-based templates, or generated once from VLM perception at task initialization. Such fixed abstractions are insuff-

ficient for open-ended and dynamic environments that demand continual adaptation. In contrast, our method evolves the symbolic world over time using feedback from symbolic validation and real execution. This enables more grounded and adaptive planning in complex embodied tasks.

3. Method

We propose **SCOPE**, a self-adaptive symbolic planning framework that enhances symbolic world modeling and long-horizon planning in embodied tasks. As shown in Figure 2, **SCOPE** actively explores the open-ended environment, constructs the symbolic world and generates an action plan (Section 3.1). The Symbolic Execution Simulator then provides feedback to evolve the symbolic world and refine the action plan (Section 3.2). **SCOPE** further distills feedback into symbolic knowledge, to enhance long-horizon planning and symbolic world modeling across tasks and scenarios (Section 3.3).

3.1. Evolving Symbolic World

In the open-ended environment, a central challenge in integrating large models with classical planners is how the symbolic world, the symbolic representations of planning problem, can accurately describe the real environment with-

out gaps, thereby supporting classical planner verification or planning. This requires symbolic world to expand along with the environment for its open-ended nature. We address this challenge with the evolving symbolic world.

As illustrated in Figure 2, the first stage of our framework constructs an initial symbolic world $\mathcal{W}_{\text{symbol}}$ from raw visual observations and task description. Within $\mathcal{W}_{\text{symbol}}$, objects are described by several types of predicates: (1) State Predicates, which capture dynamic properties of objects; (2) Property Predicates, which describe inherent object attributes; (3) Relational predicates, which express spatial and logical relationships between objects.

Based on the current symbolic world, the agent decides to either (1) explore the environment to identify additional object representations or interact with objects to enhance the understanding of object attributes. Both contribute to evolving symbolic world; or (2) generate a symbolic action plan $\mathcal{P}_A = (a_1, a_2, \dots, a_n)$, follows a PDDL format, conditioned on $\mathcal{W}_{\text{symbol}}$.

3.2. Symbolic Execution Simulator

A complete symbolic world is not derived solely from visual perception, but also from active interaction with the environment. For example, discovering that an object is not graspable only after a failed attempt to grab it. This embodied feedback reveals latent properties that are often inaccessible to vision alone. To address this, Symbolic Execution Simulator (SESim) integrates feedback from both symbolic validation and real execution to reduce the gap between the symbolic world and the open-ended environment.

As shown in Figure 2, following the construction of the symbolic world $\mathcal{W}_{\text{symbol}}$ and the generation of the initial action plan $\mathcal{P}_A$ in Section 3.1, **SCOPE** employs the Symbolic Execution Simulator to perform symbolic validation within symbolic world.

The SESim operates in two stages: (1) Symbolic Validation. The SESim utilizes the classical PDDL validator to verify the logical consistency of $\mathcal{P}_A$ within the current symbolic world $\mathcal{W}_{\text{symbol}}$, generating symbolic validation feedback $\mathcal{F}_{\text{symbol}}$. Feedback indicates logical issues such as undefined objects, unmet preconditions, or incomplete goal satisfaction. $\mathcal{F}_{\text{symbol}}$ is interpretable, and supports to refine action plan. (2) Real Execution. The SESim executes the action plan in the environment, generating real execution feedback $\mathcal{F}_{\text{real}}$. Even when the action plan passes symbolic validation, real execution failures can expose gaps between the symbolic world and open-ended environment, such as missing property predicates or incorrect relationships.

These structured feedback fields also trigger *symbolic* retrieval from SASMem. SESim forms a symbol-aligned query from $\mathcal{F}_{\text{symbol}}$ and $\mathcal{F}_{\text{real}}$ (failure signature and predicate-

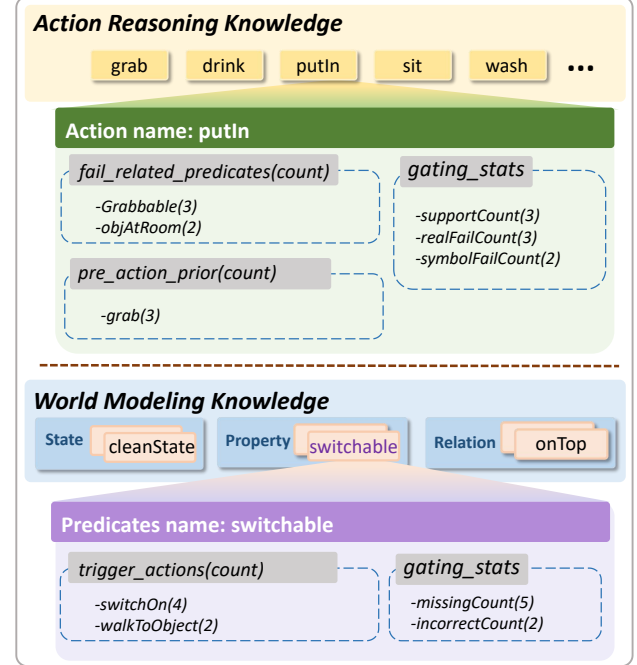


Figure 3. Overview of SASMem, illustrating its structure and symbolic knowledge components.

level mismatch evidence), retrieves the most relevant action/world entries, and aggregates them as $\mathcal{F}_{\text{SASMem}}$ to guide localized plan/world updates:

$$\mathcal{P}'_A, \mathcal{W}'_{\text{symbol}} = \text{VLM}(\mathcal{P}_A, \mathcal{F}_{\text{symbol}}, \mathcal{F}_{\text{real}}, \mathcal{F}_{\text{SASMem}})$$

SESim iterates this process, until validation succeeds or a budget is reached, thereby increasing robustness in the open-ended environment. By leveraging complementary symbolic validation and grounded feedback, **SCOPE** reduces representational gaps and inconsistencies, enabling more accurate symbolic world modeling and more reliable long-horizon planning.

3.3. Self-Adaptive Symbolic Memory

Successfully completing an embodied task in a specific environment, the agent can acquire valuable experience, such as understanding the logical structure of actions, reasoning over plans, and modeling the environment. A natural question arises: can such experience generalize to more complex tasks and environments? To support generalizable symbolic planning across diverse tasks and environments, **SCOPE** incorporates the Self-Adaptive Symbolic Memory (SASMem), a structured memory that distills predicate-level knowledge about *action reasoning* and *world modeling*.

Action reasoning knowledge captures reusable regularities of action failures and successes: for each action, it summarizes the most frequently implicated symbolic con-

ditions (e.g., missing preconditions revealed by symbolic validation) and the common successful local action patterns (i.e., which prerequisite steps tend to precede the action in successful plans).

World modeling knowledge captures persistent gaps between the symbolic world and the real environment at the predicate level. It records which predicates are commonly missing or mis-specified under grounded interaction evidence, and which interaction actions most often surface such evidence, thereby providing targeted guidance for evolving the symbolic world.

SASMem does not store raw experience directly, nor does it rely on the VLM to produce a summary; instead, it applies rule-based distillation over structured $\mathcal{F}_{\text{symbol}}$, $\mathcal{F}_{\text{real}}$ and experience to generate predicate-level memory entries. Memory updates follow a write-gating rule: an injected entry is reinforced only when it resolves the same failure pattern or pushes the first failure to a later step, and is otherwise down-weighted, with symbolic infeasibility and real-execution mismatches treated distinctly. This mechanism prevents unbounded accumulation, automatically suppresses noisy memories, and yields a continually improving memory that enhances long-horizon grounding and adaptation across tasks and environments.

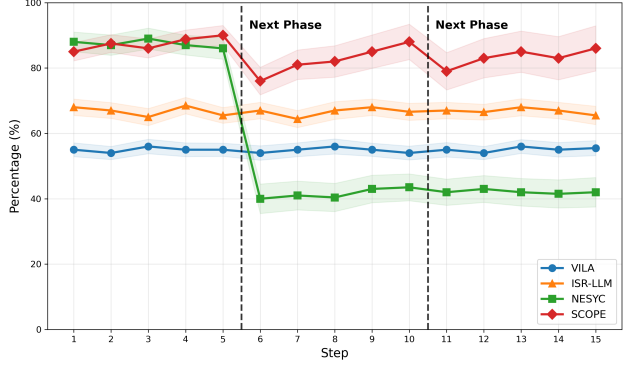
4. Experiments

Our goal is to evaluate whether **SCOPE** improves symbolic world modeling and long-horizon planning in embodied tasks under dynamic and open-ended perception. In this section, we first present the experimental setup (Section 4.1), followed by comprehensive evaluations on two key fronts: (1) grounded planning under environment perturbations (Section 4.2), (2) robust symbolic world modeling (Section 4.3). In addition, we conduct an in-depth analysis (Section 4.4), which includes: qualitative examples, ablations on the Self-Adaptive Symbolic Memory (SASMem) and Symbolic Execution Simulator (SESIM), and extensive evaluations across environments and tasks to assess the generalization and robustness of the method.

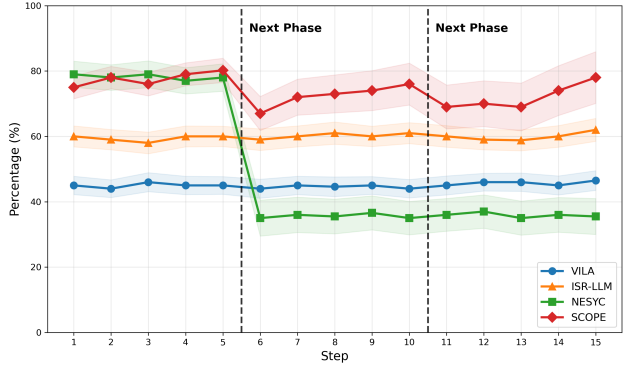
4.1. Experiment Setting

Environments. We evaluate on two embodied benchmarks, *VirtualHome* (Puig et al., 2018) and *ALFRED* (Shridhar et al., 2020). For each benchmark, we construct evaluation settings along two axes: **scene complexity** (*basic* vs. *complex*) and **environment settings** (*static*, *dynamic*, *open-ended*).

Scene complexity. *Basic* scenes feature simpler layouts with fewer interactive objects and shorter dependency chains, whereas *complex* scenes contain larger layouts with richer interactions and longer-horizon plans.



(a) VirtualHome: Per-steps StepSR on Open-ended step stream symbolic world



(b) ALFRED: Per-steps StepSR on Open-ended step stream symbolic world

Figure 4. Symbolic world evolution in open-ended settings.

Environment settings. *Static* settings keep the environment state unchanged unless affected by the agent’s actions. *Dynamic* settings introduce state changes during an episode, requiring the agent to re-ground and re-plan under non-stationary conditions. *Open-ended* settings stream a sequence of steps in which new task-required affordances or state dependencies are introduced in phases over time, requiring continual online adaptation and knowledge carry-over across tasks. The detailed construction protocol for these splits and changes is provided in the Appendix.

Baselines. We implement three baselines, categorized into two groups: (i) VLM-based planners: *VILA* (Hu et al., 2023); (ii) neuro-symbolic methods with symbolic feedback/tools: *ISR-LLM* (Zhou et al., 2024) (we use a faithful VLM-adapted re-implementation of the original LLM-based method for embodied inputs) and *NESYC* (Choi et al., 2025). For fair comparison, we use the same backbone model GPT-4o, for all VLM components in the methods.

Evaluation Metrics. We report metrics for both symbolic world quality and planning performance. For planning, we follow common practice and report *SR* (%), the percentage of tasks successfully completed, and *GC* (%), the suc-

Table 1. Planning performance (SR/GC, %) on VirtualHome and ALFRED under basic/complex scenes and static/dynamic/open-ended settings. For *open-ended* evaluation, SR/GC are averaged over the entire step stream

VirtualHome		Static		Dynamic		Open-ended	
Methods	Planner	SR	GC	SR	GC	SR	GC
Basic scenes							
VILA	VLM	55.4 $\pm$ 1.2	69.2 $\pm$ 1.9	52.3 $\pm$ 1.3	55.1 $\pm$ 1.8	47.8 $\pm$ 1.4	52.4 $\pm$ 1.9
ISR-LLM	VLM	65.8 $\pm$ 1.1	70.5 $\pm$ 1.6	61.7 $\pm$ 1.2	67.2 $\pm$ 1.5	58.1 $\pm$ 1.3	62.6 $\pm$ 1.7
NESYC	Classical planner	90.1 $\pm$ 0.7	92.7 $\pm$ 0.6	77.8 $\pm$ 1.0	80.1 $\pm$ 1.2	56.3 $\pm$ 1.0	58.9 $\pm$ 1.1
SCOPE	VLM	88.2 $\pm$ 0.8	89.4 $\pm$ 0.9	85.4 $\pm$ 0.9	86.9 $\pm$ 0.8	87.6 $\pm$ 0.8	89.2 $\pm$ 0.9
Complex scenes							
VILA	VLM	45.2 $\pm$ 1.4	46.8 $\pm$ 2.1	40.6 $\pm$ 1.5	41.4 $\pm$ 2.0	38.1 $\pm$ 1.6	40.7 $\pm$ 2.2
ISR-LLM	VLM	62.3 $\pm$ 1.3	67.1 $\pm$ 1.8	58.2 $\pm$ 1.4	62.8 $\pm$ 1.7	54.5 $\pm$ 1.5	56.0 $\pm$ 1.9
NESYC	Classical planner	84.3 $\pm$ 0.8	85.5 $\pm$ 0.7	72.4 $\pm$ 1.2	77.7 $\pm$ 1.4	50.9 $\pm$ 1.2	53.5 $\pm$ 1.3
SCOPE	VLM	86.5 $\pm$ 1.0	88.8 $\pm$ 1.1	82.1 $\pm$ 1.0	85.6 $\pm$ 0.9	79.4 $\pm$ 0.9	82.0 $\pm$ 1.0

ALFRED		Static		Dynamic		Open-ended	
Methods	Planner	SR	GC	SR	GC	SR	GC
Basic scenes							
VILA	VLM	50.7 $\pm$ 1.3	57.4 $\pm$ 2.0	47.2 $\pm$ 1.4	50.8 $\pm$ 1.9	44.3 $\pm$ 1.5	45.9 $\pm$ 2.0
ISR-LLM	VLM	63.2 $\pm$ 1.2	79.1 $\pm$ 1.7	58.6 $\pm$ 1.3	61.5 $\pm$ 1.6	61.8 $\pm$ 1.4	64.3 $\pm$ 1.8
NESYC	Classical planner	88.4 $\pm$ 0.7	90.1 $\pm$ 0.6	71.7 $\pm$ 1.1	75.2 $\pm$ 1.3	50.1 $\pm$ 1.1	53.5 $\pm$ 1.2
SCOPE	VLM	85.8 $\pm$ 0.9	89.1 $\pm$ 1.0	83.1 $\pm$ 0.9	85.7 $\pm$ 0.8	80.8 $\pm$ 0.8	84.4 $\pm$ 0.9
Complex scenes							
VILA	VLM	36.5 $\pm$ 1.5	43.2 $\pm$ 2.2	33.8 $\pm$ 1.6	37.1 $\pm$ 2.1	30.9 $\pm$ 1.7	32.2 $\pm$ 2.3
ISR-LLM	VLM	59.8 $\pm$ 1.4	63.6 $\pm$ 1.9	55.3 $\pm$ 1.5	58.0 $\pm$ 1.8	54.4 $\pm$ 1.6	56.8 $\pm$ 2.0
NESYC	Classical planner	82.8 $\pm$ 0.8	83.6 $\pm$ 0.7	64.3 $\pm$ 1.3	67.8 $\pm$ 1.5	51.7 $\pm$ 1.3	52.1 $\pm$ 1.4
SCOPE	VLM	78.4 $\pm$ 1.1	80.5 $\pm$ 1.2	76.9 $\pm$ 1.0	78.4 $\pm$ 0.9	75.6 $\pm$ 1.0	77.2 $\pm$ 1.1

cess rate of individual goal conditions. For open-ended step-stream evaluation, we additionally report *StepSR* (%), Step Success Rate, defined as the step-level success rate of executed actions in the environment: it measures the percentage of action steps that are successfully carried out, averaged over the entire step stream. For symbolic world modeling, we measure *ClassicalSR* (%), the fraction of tasks whose constructed symbolic world is sufficiently complete/consistent for a classical planner to produce a valid plan; *SymRecall* (%), the recall of benchmark-/expert-provided ground-truth task-relevant predicates in the symbolic world.

4.2. Planning under Environment Settings

Robust planning across settings. Table 1 compares long-horizon planning under *static*, *dynamic*, and *open-ended* settings. In *static* environments, the symbolic world is relatively complete and stable, so methods that rely on a classical planner as the planner can leverage exact symbolic search to produce causally grounded action plans. However, this advantage diminishes in *dynamic* settings, where non-stationary state changes quickly make the initial sym-

bolic snapshot stale. Once key predicates become missing or outdated, classical planners tend to fail and crucially cannot effectively inform the VLM which specific predicates or state facts are absent. By contrast, the VLM can still propose actions, while symbolic validation and execution feedback can be used to pinpoint violated preconditions and mismatches, enabling targeted correction. This brittleness is further amplified in *open-ended* episode streams, where newly emerging affordances and state dependencies systematically exceed the initial symbolic coverage.

Open-ended Step-Stream Results. We report *StepSR* (%), *Step Success Rate*, which measures whether each executed action step succeeds in the environment over the open-ended stream. *VILA* (VLM-only planner) remains largely flat across the stream, with only minor fluctuations, yet consistently exhibits weak step executability. *ISR-LLM* shows similarly limited variation across phases; while it benefits from validator-provided symbolic error signals, symbolic feedback alone is often insufficient to bridge the symbolic-real gap under continual novelty. In contrast, *NESYC* (classical planner) performs strongly early on but degrades sharply after phase transitions, indicating brittleness when newly in-

Table 2. Symbolic world quality measured by *ClassicalSR* and *SymRecall* on both benchmarks, averaged over *open-ended* settings.

Method	Evolve World?	ClassicalSR		SymRecall	
		ALFRED	VirtualHome	ALFRED	VirtualHome
Basic scenes					
SCOPE	✓	94.8 ± 0.8	95.8 ± 0.7	69.2 ± 2.8	65.5 ± 3.1
w/o SESim	✗	72.3 ± 3.9	74.2 ± 1.2	52.1 ± 3.5	53.3 ± 3.7
w/o SASMem	✓	80.2 ± 1.8	85.3 ± 0.9	54.1 ± 3.2	50.2 ± 3.4
w/o SESim & SASMem	✗	70.0 ± 5.4	72.8 ± 1.6	50.1 ± 4.8	48.1 ± 4.9
Complex scenes					
SCOPE	✓	89.0 ± 1.0	90.8 ± 0.8	52.3 ± 4.2	64.1 ± 3.5
w/o SESim	✗	64.2 ± 5.1	74.3 ± 3.3	45.4 ± 4.8	51.2 ± 4.6
w/o SASMem	✓	70.2 ± 1.4	75.6 ± 1.5	44.9 ± 4.7	48.9 ± 4.5
w/o SESim & SASMem	✗	45.2 ± 5.8	61.6 ± 5.9	37.6 ± 5.5	40.7 ± 5.0

roduced affordances or dependencies exceed the initial symbolic coverage. **SCOPE** achieves the most robust behavior: although it may exhibit transient drops near phase boundaries, it rapidly recovers in subsequent steps, demonstrating effective online adaptation under open-ended changes.

Planning over long horizons and increasing scene complexity. We further compare *basic* versus *complex* scenes to stress-test scalability under longer action horizons and richer object interactions. While baselines tend to degrade as complexity increases due to compounding grounding errors and limited symbolic coverage, **SCOPE** remains more robust by continuously updating symbolic predicates and revising plan steps based on newly observed evidence during execution. Overall, these results suggest that **SCOPE** not only adapts to changing and open-ended environments, but also better sustains coherent and grounded long-horizon planning as scene complexity grows.

4.3. Symbolic World Evaluation

Symbolic worlds constructed directly by a VLM in *open-ended* settings are often incomplete or inconsistent, resulting in low *ClassicalSR* and fragile long-horizon planning (Table 2). Methods without symbolic world evolution remain constrained by a static world model and struggle to keep pace with newly introduced affordances and state dependencies over the stream. In contrast, *SESim* leverages feedback from symbolic validation and real execution to iteratively evolve the symbolic world, correcting missing predicates and faulty preconditions, which substantially improves *ClassicalSR*, especially in complex scenes. We further assess semantic alignment with *SymRecall*: one-shot perception commonly misses object attributes and relations, whereas feedback-driven refinement uncovers latent properties revealed through interaction (e.g., non-graspable objects), underscoring the importance of interaction-informed world evolution under continual novelty.

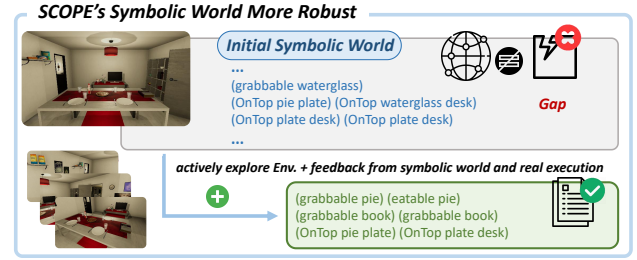


Figure 5. Symbolic world evolution example.

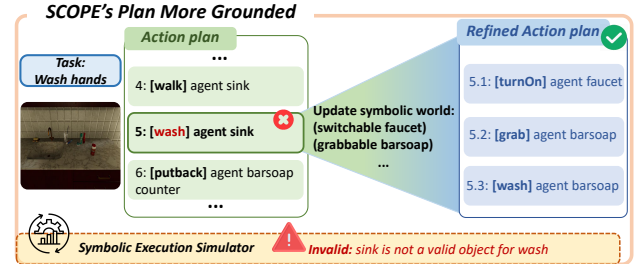


Figure 6. Action plan refinement example.

4.4. In-Depth Analysis and Generalization

Qualitative Analysis. We qualitatively examine how **SCOPE** enhances symbolic modeling and planning under environment perturbations, shown in Figure 5 and Figure 6. Compared to direct VLM outputs, **SCOPE** produces more complete symbolic world by feedback. This leads to better recovery of missing object attributes and relational representations. In planning, symbolic validation allows **SCOPE** to detect and resolve causal gaps, resulting in more grounded and long-horizon action plan.

Analysis of Symbolic Execution Simulator. We assess the respective roles of symbolic validation and real execution feedback within SESim (Table 3). Using only real execution

Table 3. Ablation on SESim feedback sources on *VirtualHome*. Impact of using real execution feedback only, symbolic validation only, or both on *ClassicalSR*, *SymRecall*, and *GC*.

Method	<i>ClassicalSR</i>	<i>SymRecall</i>	<i>GC</i>
SESim(Real exec. only)	80.2 $\pm$ 1.8	55.6 $\pm$ 2.4	81.6 $\pm$ 1.7
SESim(Validation only)	86.5 $\pm$ 1.5	57.1 $\pm$ 2.1	80.3 $\pm$ 1.6
SESim(Full)	92.6 $\pm$ 1.2	62.6 $\pm$ 1.8	82.1 $\pm$ 1.3

Table 4. Average plan/world update counts on *VirtualHome* under different memory designs (*#PlanRefine*/*#EnvEvolve*).

Method	<i>#PlanRefine</i>	<i>#EnvEvolve</i>
RawMem	6.51 $\pm$ 0.45	3.91 $\pm$ 0.33
SummMem	4.21 $\pm$ 0.35	2.02 $\pm$ 0.24
SASMem	2.18 $\pm$ 0.28	0.86 $\pm$ 0.18

feedback allows the agent to detect that an action has failed and that a mismatch exists between the symbolic world and the real environment, but it cannot identify the underlying cause—whether the failure arises from a missing object or from incorrect object properties. Conversely, relying on symbolic validation exposes logical error in the plan, such as unmet preconditions or missing symbolic entities. However, evolving the symbolic world based only on symbolic validation can inadvertently enlarge the gap between symbolic representations and open-ended environment. Combined, they provide complementary feedback that jointly guide more accurate symbolic world evolution and more robust planning.

Analysis of Self-Adaptive Symbolic Memory. We analyze the impact of SASMem by comparing it with two generic memory-augmented baselines, *RawMem* and *SummMem*, which store and retrieve past experience to augment the VLM prompt without structured, predicate-level distillation (Table 4). Specifically, *RawMem* retrieves raw traces, while *SummMem* retrieves VLM-produced summaries. In contrast, SASMem organizes feedback from symbolic validation and real execution in a predicate-level, symbol-aligned form with higher information density, enabling more efficient and precise guidance—reflected by fewer plan refinements and symbolic world updates.

Symbolic Hallucination Evaluation. We measure *SymHalluc* (%), defined as the fraction of introduced/refined task-relevant predicates that are inconsistent with the ground truth, under basic and complex environments (Table 5). SASMem contributes to symbolic grounding through adaptive knowledge prompting, its ability to suppress hallucinated predicates remains limited, particularly when symbolic world diverge from the open-ended environments. In contrast, SESim plays a key role in mitigating symbolic hallucination by leveraging feedback from symbolic validation and real execution. This feedback helps identify and

Table 5. Impact of SESim and SASMem on *SymHalluc*(%).

Method	<i>SymHalluc</i> (%)	
	<i>VirtualHome</i>	ALFRED
SCOPE	3.68 $\pm$ 0.6	5.50 $\pm$ 0.8
w/o SASMem	5.72 $\pm$ 1.3	7.91 $\pm$ 1.4
w/o SESim	9.88 $\pm$ 1.9	10.43 $\pm$ 2.0
w/o SASMem & SESim	14.01 $\pm$ 2.6	16.28 $\pm$ 3.1

Table 6. Impact of different backbones on planning and symbolic world modeling.

Method	SR	GC	ClassicalSR	SymRecall
Qwen3-VL-8B				
ISR-LLM	57.7 $\pm$ 1.1	63.2 $\pm$ 1.4	67.2 $\pm$ 2.8	53.1 $\pm$ 2.7
SCOPE	76.8 $\pm$ 2.3	78.7 $\pm$ 1.7	89.3 $\pm$ 2.5	59.6 $\pm$ 3.2
GPT-4o				
ISR-LLM	65.2 $\pm$ 2.2	70.5 $\pm$ 1.8	70.4 $\pm$ 2.7	52.5 $\pm$ 2.2
SCOPE	86.5 $\pm$ 1.3	88.5 $\pm$ 2.1	96.5 $\pm$ 1.5	63.5 $\pm$ 1.3

revise symbolic hallucination, progressively narrowing the gap between the symbolic world and the real environment.

Impact of Different VLMs. We further examine backbone sensitivity on *VirtualHome*, and observe consistent trends across benchmarks. Table 6 shows that **SCOPE** consistently outperforms ISR-LLM across both backbones on planning performance (SR/GC) as well as symbolic world quality (ClassicalSR/SymRecall). Notably, the advantage persists with the smaller backbone (Qwen3-VL-8B), suggesting that the improvements are driven by our feedback-grounded closed-loop—i.e., iteratively evolving the symbolic world and refining plans using symbolic validation and real execution—rather than relying solely on model scale. When switching to a stronger backbone (GPT-4o), both methods improve in absolute terms, yet **SCOPE** retains a clear margin, indicating that the proposed refinement and memory distillation mechanisms generalize across backbone choices and further benefit from increased model capacity.

5. Conclusion

We present **SCOPE**, a self-adaptive symbolic planning framework that evolves symbolic world representations through feedback from both symbolic validation and real execution. By closing the loop between perception, reasoning, and interaction, **SCOPE** addresses the limitations of static symbolic modeling and non-adaptive symbolic knowledge in open-ended environments, providing a general way to integrate large models with classical planners. Experiments demonstrate that evolving symbolic representations leads to more grounded long-horizon planning, and better generalization across tasks and environments.

Acknowledgements

This work was supported by the National Key Research and Development Program of China (2025ZD0123100), National Natural Science Foundation of China (62436007), Zhejiang NSF (LQK26F020001), Key R&D Program of Zhejiang (2026SDXT005), Ningbo Yongjiang Talent Introduction Programme (2024A-401-G), Zhejiang University Education Foundation Qizhen Scholar Foundation.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- Aeronautiques, C., Howe, A., Knoblock, C., McDermott, I. D., Ram, A., Veloso, M., Weld, D., Sri, D. W., Barrett, A., Christianson, D., et al. Pddl—the planning domain definition language. *Technical Report, Tech. Rep.*, 1998.
- Ahn, M., Brohan, A., Brown, N., Chebotar, Y., Cortes, O., David, B., Finn, C., Fu, C., Gopalakrishnan, K., Hausman, K., et al. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022.
- Akakzia, A., Colas, C., Oudeyer, P.-Y., Chetouani, M., and Sigaud, O. Grounding language to autonomously-acquired skills via goal generation. *arXiv preprint arXiv:2006.07185*, 2020.
- Alayrac, J.-B., Donahue, J., Luc, P., Miech, A., Barr, I., Hasson, Y., Lenc, K., Mensch, A., Millican, K., Reynolds, M., et al. Flamingo: a visual language model for few-shot learning. *Advances in neural information processing systems*, 35:23716–23736, 2022.
- Ao, S., Salim, F. D., and Khan, S. Emac+: Embodied multimodal agent for collaborative planning with vlm+ llm. *arXiv preprint arXiv:2505.19905*, 2025.
- Chen, B., Xu, Z., Kirmani, S., Ichter, B., Sadigh, D., Guibas, L., and Xia, F. Spatialvlm: Endowing vision-language models with spatial reasoning capabilities. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 14455–14465, 2024.
- Cheng, A.-C., Yin, H., Fu, Y., Guo, Q., Yang, R., Kautz, J., Wang, X., and Liu, S. Spatialrgpt: Grounded spatial reasoning in vision-language models. *Advances in Neural Information Processing Systems*, 37:135062–135093, 2024.
- Choi, W., Park, J., Ahn, S., Lee, D., and Woo, H. Nesyc: A neuro-symbolic continual learner for complex embodied tasks in open domains. *arXiv preprint arXiv:2503.00870*, 2025.
- Driess, D., Xia, F., Sajjadi, M. S. M., Lynch, C., Chowdhery, A., Ichter, B., Wahid, A., Tompson, J., Vuong, Q., Yu, T., Huang, W., Chebotar, Y., Sermanet, P., Duckworth, D., Levine, S., Vanhoucke, V., Hausman, K., Toussaint, M., Greff, K., Zeng, A., Mordatch, I., and Florence, P. Palm-e: An embodied multimodal language model, 2023. URL <https://arxiv.org/abs/2303.03378>.
- Du, Y., Liu, Z., Li, J., and Zhao, W. X. A survey of vision-language pre-trained models. *arXiv preprint arXiv:2202.10936*, 2022.
- Dwivedi, R., Dave, D., Naik, H., Singhal, S., Omer, R., Patel, P., Qian, B., Wen, Z., Shah, T., Morgan, G., et al. Explainable ai (xai): Core ideas, techniques, and solutions. *ACM computing surveys*, 55(9):1–33, 2023.
- Fei, H., Wu, S., Zhang, H., Chua, T.-S., and Yan, S. Vitrion: A unified pixel-level vision llm for understanding, generating, segmenting, editing. *Advances in neural information processing systems*, 37:57207–57239, 2024.
- Fung, P., Bachrach, Y., Celikyilmaz, A., Chaudhuri, K., Chen, D., Chung, W., Dupoux, E., Jégou, H., Lazaric, A., Majumdar, A., et al. Embodied ai agents: Modeling the world. *arXiv preprint arXiv:2506.22355*, 2025.
- Hu, M., Chen, T., Zou, Y., Lei, Y., Chen, Q., Li, M., Mu, Y., Zhang, H., Shao, W., and Luo, P. Text2world: Benchmarking large language models for symbolic world model generation. *arXiv preprint arXiv:2502.13092*, 2025.
- Hu, Y., Lin, F., Zhang, T., Yi, L., and Gao, Y. Look before you leap: Unveiling the power of gpt-4v in robotic vision-language planning. *arXiv preprint arXiv:2311.17842*, 2023.
- Huang, W., Abbeel, P., Pathak, D., and Mordatch, I. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In *International conference on machine learning*, pp. 9118–9147. PMLR, 2022a.
- Huang, W., Xia, F., Xiao, T., Chan, H., Liang, J., Florence, P., Zeng, A., Tompson, J., Mordatch, I., Chebotar, Y., et al. Inner monologue: Embodied reasoning through planning with language models. *arXiv preprint arXiv:2207.05608*, 2022b.

- Hurst, A., Lerer, A., Goucher, A. P., Perelman, A., Ramesh, A., Clark, A., Ostrow, A., Welihinda, A., Hayes, A., Radford, A., et al. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*, 2024.
- Jiang, Y., Gu, S. S., Murphy, K. P., and Finn, C. Language as an abstraction for hierarchical deep reinforcement learning. *Advances in neural information processing systems*, 32, 2019a.
- Jiang, Y.-q., Zhang, S.-q., Khandelwal, P., and Stone, P. Task planning in robotics: an empirical comparison of pddl-and asp-based systems. *Frontiers of Information Technology & Electronic Engineering*, 20(3):363–373, 2019b.
- Liang, J., Huang, W., Xia, F., Xu, P., Hausman, K., Ichter, B., Florence, P., and Zeng, A. Code as policies: Language model programs for embodied control. *arXiv preprint arXiv:2209.07753*, 2022.
- Liu, B., Jiang, Y., Zhang, X., Liu, Q., Zhang, S., Biswas, J., and Stone, P. Llm+ p: Empowering large language models with optimal planning proficiency. *arXiv preprint arXiv:2304.11477*, 2023.
- Lu, C., Krishna, R., Bernstein, M., and Fei-Fei, L. Visual relationship detection with language priors. In *European conference on computer vision*, pp. 852–869. Springer, 2016.
- Mao, J., Gan, C., Kohli, P., Tenenbaum, J. B., and Wu, J. The neuro-symbolic concept learner: Interpreting scenes, words, and sentences from natural supervision. *arXiv preprint arXiv:1904.12584*, 2019.
- Migimatsu, T. and Bohg, J. Grounding predicates through actions. In *2022 International Conference on Robotics and Automation (ICRA)*, pp. 3498–3504. IEEE, 2022.
- Mirchandani, S., Karamcheti, S., and Sadigh, D. Ella: Exploration through learned language abstraction. *Advances in neural information processing systems*, 34: 29529–29540, 2021.
- Nair, S. and Finn, C. Hierarchical foresight: Self-supervised learning of long-horizon tasks via visual subgoal generation. *arXiv preprint arXiv:1909.05829*, 2019.
- Puig, X., Ra, K., Boben, M., Li, J., Wang, T., Fidler, S., and Torralba, A. Virtualhome: Simulating household activities via programs. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 8494–8502, 2018.
- Sarch, G., Jang, L., Tarr, M., Cohen, W. W., Marino, K., and Fragkiadaki, K. Vlm agents generate their own memories: Distilling experience into embodied programs of thought. *Advances in Neural Information Processing Systems*, 37: 75942–75985, 2024.
- Shridhar, M., Thomason, J., Gordon, D., Bisk, Y., Han, W., Mottaghi, R., Zettlemoyer, L., and Fox, D. Alfred: A benchmark for interpreting grounded instructions for everyday tasks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 10740–10749, 2020.
- Silver, T., Hariprasad, V., Shuttleworth, R. S., Kumar, N., Lozano-Pérez, T., and Kaelbling, L. P. Pddl planning with pretrained large language models. In *NeurIPS 2022 foundation models for decision making workshop*, 2022.
- Silver, T., Chitnis, R., Kumar, N., McClinton, W., Lozano-Pérez, T., Kaelbling, L., and Tenenbaum, J. B. Predicate invention for bilevel planning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pp. 12120–12129, 2023.
- Singh, I., Blukis, V., Mousavian, A., Goyal, A., Xu, D., Tremblay, J., Fox, D., Thomason, J., and Garg, A. Prog-prompt: program generation for situated robot task planning using large language models. *Autonomous Robots*, 47(8):999–1012, 2023.
- Srinivas, A., Jabri, A., Abbeel, P., Levine, S., and Finn, C. Universal planning networks: Learning generalizable representations for visuomotor control. In *International conference on machine learning*, pp. 4732–4741. PMLR, 2018.
- Wong, L., Mao, J., Sharma, P., Siegel, Z. S., Feng, J., Korneev, N., Tenenbaum, J. B., and Andreas, J. Learning adaptive planning representations with natural language guidance. *arXiv preprint arXiv:2312.08566*, 2023.
- Wu, Q., Zhao, H., Saxon, M., Bui, T., Wang, W. Y., Zhang, Y., and Chang, S. Vsp: Assessing the dual challenges of perception and reasoning in spatial planning tasks for vlms. *arXiv preprint arXiv:2407.01863*, 2024.
- Xiong, S., Zhou, J., Liu, Z., and Su, Y. Symplanner: Deliberate planning in language models with symbolic representation. *arXiv preprint arXiv:2505.01479*, 2025.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Yang, X., Wu, Y., Yang, M., Chen, H., and Geng, X. Exploring diverse in-context configurations for image captioning. *Advances in Neural Information Processing Systems*, 36:40924–40943, 2023.

Zeng, A., Attarian, M., Ichter, B., Choromanski, K., Wong, A., Welker, S., Tombari, F., Purohit, A., Ryoo, M., Sindhwani, V., et al. Socratic models: Composing zero-shot multimodal reasoning with language. *arXiv preprint arXiv:2204.00598*, 2022.

Zhang, C., Li, Z., and Yuan, W. Vote-tree-planner: Optimizing execution order in llm-based task planning pipeline via voting. *arXiv preprint arXiv:2502.09749*, 2025.

Zhang, X., Ding, Y., Amiri, S., Yang, H., Kaminski, A., Esselink, C., and Zhang, S. Grounding classical task planners via vision-language models. *arXiv preprint arXiv:2304.08587*, 2023.

Zhi-Xuan, T. *Pddl. jl: An extensible interpreter and compiler interface for fast and flexible ai planning*. PhD thesis, Massachusetts Institute of Technology, 2022.

Zhou, Z., Song, J., Yao, K., Shu, Z., and Ma, L. Isr-llm: Iterative self-refined large language model for long-horizon sequential task planning. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 2081–2088. IEEE, 2024.

Zhu, Y., Tremblay, J., Birchfield, S., and Zhu, Y. Hierarchical planning for long-horizon manipulation with geometric and symbolic scene graphs. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 6541–6548. Ieee, 2021.

Appendix

A. Limitations and Future Discussions

While **SCOPE** demonstrates improvements in symbolic planning and long-horizon planning, there are several limitations that warrant further exploration. In this section, we discuss the current limitations of our framework and outline potential directions for future work.

A.1. Limitations

Real-World Execution Feedback. In a simulator, real execution feedback is directly provided by the program, such as when an action cannot be executed or when an object is not found. However, in the real world, feedback from real execution is more difficult to describe. Unlike in a simulated environment where actions and object states are explicitly programmed, real-world feedback involves sensory inputs, environmental interactions, and physical constraints that are not always easily translated into direct feedback or error messages. This complexity requires sophisticated sensor integration and real-time processing to accurately reflect the outcomes of actions.

A.2. Future Directions

Integration with Reinforcement Learning (RL). Future work could explore integrating **SCOPE** with RL to enhance the agent’s ability to learn policies for symbolic task planning. By combining symbolic planning with RL, the agent may not only refine its symbolic world representations but also optimize long-horizon planning by exploring more complex policies and rewards.

Expanding to Multi-Agent Environments. **SCOPE** could be extended to multi-agent environments, where the complexity of planning increases significantly. PDDL has the ability to model complex multi-agent interactions, and integrating it into **SCOPE** could enable the system to handle planning scenarios that involve multiple agents with distinct goals and constraints. This would make **SCOPE** more applicable to collaborative robotics and other multi-agent systems.

B. Evaluation Setup

B.1. Task Category Specification

We summarize the average symbolic action length and the number of involved environment objects for each task category in Table 7.

VirtualHome. We use a subset of $n = 186$ tasks randomly sampled from the *VirtualHome* dataset. To assess the generalization of symbolic planning across diverse tasks and semantic domains, we further organize these tasks into four

Table 7. Average number of symbolic actions and objects in the environment involved in the scenes in *VirtualHome* and *ALFRED*.

TaskName	#Actions (avg.)	#Objects (avg.)
VirtualHome		
Personal Hygiene	4.3	51.6
Food and Dining	7.2	66.8
Housekeeping	6.3	54.5
Leisure	5.3	33.7
ALFRED		
Pick & Place	16.2	42.2
Examine in Light	8.7	37.7
Clean & Place	18.1	57.1
Heat & Place	11.2	69.6
Cool & Place	20.4	80.6
Pick Two & Place	14.8	56.8

manually defined categories based on their object dependencies. (1) *Personal Hygiene*: Brush teeth, Wash hands, Go to toilet (2) *Food and Dining*: Cook some food, Drink, Set up table, Put groceries in fridge (3) *Housekeeping*: Wash clothes, Wash dishes by hand, Wash dishes with dishwasher, Turn on lightswitch (4) *Leisure*: Watch TV, Change TV channel, Listen to music, Read book, Relax on sofa, Pick up phone, Browse internet

ALFRED. We use a subset of $n = 600$ tasks randomly sampled from the *ALFRED* dataset. These tasks are selected from the following six categories: (1) *Pick & Place*, (2) *Pick Two & Place*, (3) *Examine in Light*, (4) *Clean & Place*, (5) *Heat & Place*, and (6) *Cool & Place*. From each of these categories, we randomly select 100 tasks, ensuring a diverse set of scenarios to evaluate **SCOPE**. These tasks test the method’s ability to handle complex action sequences and to manage multiple objects in open-ended environments.

B.2. Open-ended Setting Construction

The open-ended setting is designed to evaluate adaptation when (i) the task distribution shifts over time and (ii) task-required affordances/state dependencies emerge beyond the agent’s initial symbolic coverage. Unlike the static/dynamic settings that evaluate each task independently, the open-ended setting evaluates a stream of tasks with phased novelty, where knowledge can be carried over across tasks.

Phase-based task stream. We construct an ordered task stream $\{\tau_1, \tau_2, \dots, \tau_K\}$ and partition it into $K = 3$ consecutive phases. Each phase corresponds to a task category (Appendix B.1) and is intended to introduce a new category-specific novelty. Within a phase, tasks are randomly permuted; across phases, the phase order is fixed for a given stream seed. For open-ended evaluation, we execute tasks sequentially and *do not reset* the agent’s symbolic knowl-

Table 8. Schema of SASMem.

Field	Description
Action reasoning knowledge	
action name	The target action that this entry describes.
fail related predicates	A multiset predicate count of predicates frequently implicated when <i>action_name</i> fails.
support count	Number of times retrieving this entry leads to the same failure pattern being resolved (or the first failure pushed to a later step).
real fail count	Number of times the plan still fails at real execution after using this entry.
symbol fail count	Number of times the plan still fails in symbolic validation after using this entry.
pre action prior	A sequence prior extracted from successful plans: we count which action most frequently occurs immediately before <i>action_name</i> .
World modeling knowledge	
predicate name	The predicate that this entry models.
trigger action	Action count indicating which interaction actions most often trigger grounded evidence or real failures related to <i>predicate_name</i> (typically derived from <i>fail_action</i>).
missing count	Number of times grounded execution evidence indicates <i>predicate_name</i> is required but missing from symbolic world.
incorrect count	Number of times grounded execution evidence indicates symbolic world contains an incorrect instance/value of <i>predicate_name</i> .

edge/memory between tasks (the environment is reset per task as in standard evaluation, but the agent’s internal state persists), enabling knowledge carry-over across the stream.

Category-specific new action/affordance injection. To simulate the introduction of novel actions in an open-ended manner, we associate each task category c_k (the k -th phase) with a *category-specific* symbolic action template $a_{\text{new}}^{(k)}$ that represents a new affordance or a new state dependency required by tasks in that category. We implement this novelty by *gating* the availability of action schemas (and their predicate interfaces) across phases: at initialization (phase 1), the agent is only provided with a core set of generic actions (e.g., navigation/manipulation primitives and common interaction actions). At the beginning of phase k , the environment contains tasks whose successful completion *requires* at least one instance of $a_{\text{new}}^{(k)}$ (or its associated predicate-level dependency), while this action template is absent from the agent’s initial symbolic coverage.

B.3. Basic vs. Complex Environments.

For both *VirtualHome* and *ALFRED*, we further partition tasks into *basic* and *complex* environments in order to analyze how **SCOPE** behaves under different levels of environment complexity, as summarized in Table 10.

For *VirtualHome*, we define complexity based on the spatial extent of the scene explored by the agent. During execution,

we record the set of distinct rooms that the agent visits while completing the task. If the task can be accomplished within a single room, we label it as belonging to the *basic* environment. If the agent needs to visit multiple rooms to complete the task, we label it as belonging to the *complex* environment. The average symbolic action sequence length and the average number of environment objects involved per task for each category are summarized in Table 7.

For *ALFRED*, we define complexity based on the number of objects in the environment involved in the scenes. We label a task as *basic* if this number is at most 30, and as *complex* otherwise. The threshold of 30 is chosen such that both splits contain a sufficient number of tasks while the *complex* split exhibits significantly higher object cardinality.

B.4. Evaluation Metrics

We provide precise definitions for all evaluation metrics used in our experiments:

- **Step Success Rate (StepSR):** The step-level success rate of actions executed in the environment, averaged over the entire open-ended step stream:

$$\text{StepSR} = \frac{N_{\text{exec_succ}}}{N_{\text{exec}}}$$

where N_{exec} is the total number of action steps actually

Table 9. Structured feedback schema used by SESim.

Field	Description
Symbolic validation feedback	
fail step	Index of the first failing step in the action sequence during symbolic validation.
fail action	Name of the failing action.
error type	Error category returned by the PDDL validator, e.g., <i>unmet_precondition</i> , <i>undefined_object</i> , <i>goal_not_satisfied</i> , ...
error content	Detailed error content. For <i>unmet_precondition</i> , it lists the missing predicate instances (e.g., <i>Graspable</i>).
Real execution feedback	
outcome	Execution outcome of the attempted plan in the environment: success or fail.
fail content	Failure reason reported by the environment e.g., <i>not_found</i> , <i>not_reachable</i> , ...

Table 10. Average number of symbolic actions and distinct interactive objects per task, as well as the number of tasks in each group.

task	#Actions (avg.)	#Objects (avg.)	#Tasks
VirtualHome			
basic	5.3	34.7	57
complex	7.9	82.9	129
ALFRED			
basic	12.8	29.3	227
complex	15.9	68.9	273

executed in the environment over the step stream, and $N_{\text{exec_succ}}$ is the number of those steps that succeed.

- **Classical Planner Support Rate (ClassicalSR):** The percentage of tasks in which the symbolic world is sufficiently complete to allow a classical planner to generate a valid plan:

$$\text{ClassicalSR} = \frac{N_{\text{supported}}}{N_{\text{total}}}$$

where $N_{\text{supported}}$ is the number of tasks with planner-supported symbolic models, and N_{total} is the total number of tasks.

- **Task-Relevant Predicate Recall (SymRecall):** The percentage of ground-truth task-relevant predicates correctly recovered in the symbolic world:

$$\text{SymRecall} = \frac{|\mathcal{P}_{\text{pred}} \cap \mathcal{P}_{\text{gt}}|}{|\mathcal{P}_{\text{gt}}|}$$

where $\mathcal{P}_{\text{gt}}$ is the set of ground-truth predicates relevant to task completion (e.g., goal achievement, action pre-

conditions), and $\mathcal{P}_{\text{pred}}$ is the corresponding predicted set.

- **Symbolic Hallucination Rate (SymHalluc):** The percentage of predicted task-relevant predicates that do not appear in the ground-truth symbolic world:

$$\text{SymHalluc} = \frac{|\mathcal{P}_{\text{pred}} \setminus \mathcal{P}_{\text{gt}}|}{|\mathcal{P}_{\text{pred}}|}$$

This measures the extent to which the model generates incorrect symbolic information related to the task, including hallucinated object states or spatial relations.

C. Symbolic Execution Simulator Details

Figure 7 provides a qualitative example of the Symbolic Execution Simulator (SESim). Starting from an initial symbolic world and plan, SESim first performs symbolic validation to detect logical inconsistencies and then executes the plan in the simulator to obtain grounded feedback, which together are used to refine action plan and evolve the symbolic world.

C.1. Symbolic Validation

The symbolic validation is implemented by wrapping an off-the-shelf PDDL validator. We use a standard VAL-style PDDL validator in all experiments. Given a symbolic world represented as a PDDL domain file and a problem file, together with a PDDL plan, the validator checks whether the plan is logically consistent with the current symbolic world. Concretely, the validator expands the plan step by step and verifies that, for each action: (i) all preconditions are satisfied in the current symbolic state, (ii) all referenced objects are defined and correctly typed, and (iii) the final state satisfies the goal conditions specified in the problem file.

If any of these checks fail, the validator returns a structured error trace containing unsatisfied preconditions, undefined or misspecified objects, and unmet goal predicates. We parse this trace into the symbolic validation feedback, which includes: (1) the index of the failing action, (2) the missing or violated predicates, and (3) the subset of goals that remain unsatisfied.

C.2. Real Execution

In the real execution, the same PDDL plan is executed step-by-step in the simulator environment. Because the PDDL actions are defined at a high level with explicit preconditions and effects, we design the symbolic domain such that each PDDL action corresponds closely to a primitive executable action in the simulator. This allows us to directly map each action to an environment action and apply it to the current simulated state.

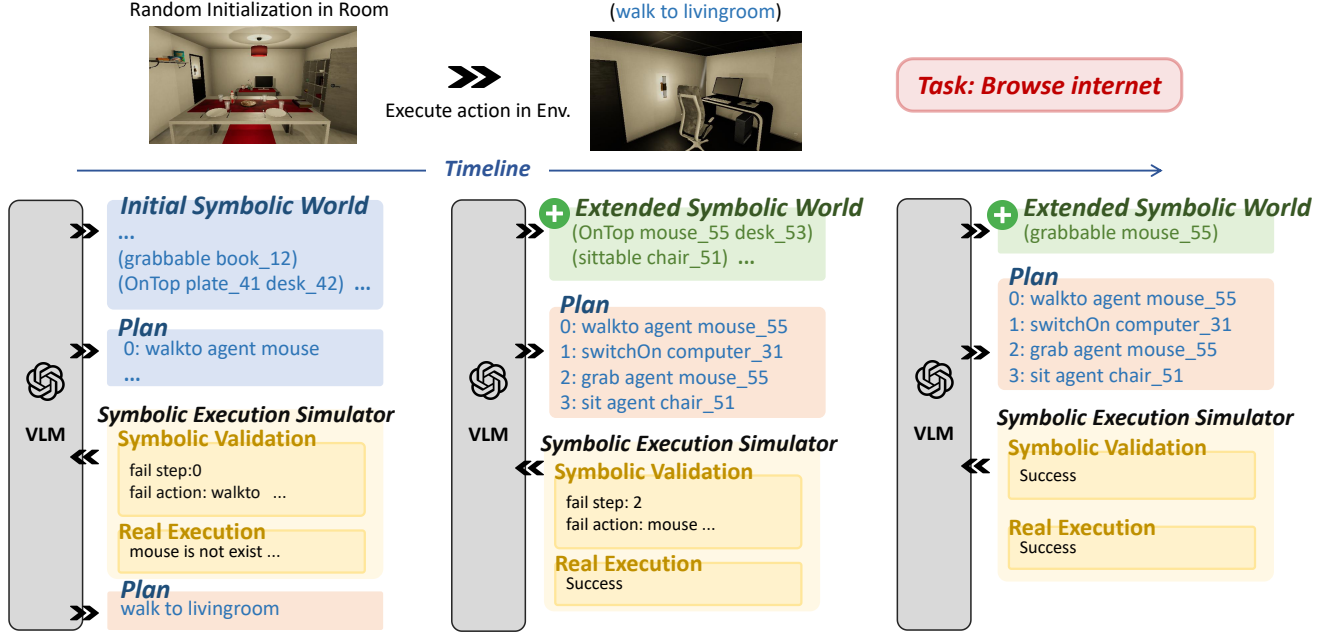


Figure 7. Example of how the Symbolic Execution Simulator (SESIm) jointly uses symbolic validation and real execution feedback to refine both the action plan and the symbolic world.

For *VirtualHome*, we align PDDL actions with the built-in program-based API (e.g., Walk, Grab, SwitchOn), and execute them sequentially in the environment. The simulator reports whether an action succeeds, fails because the target object cannot be found, or fails because the action is not applicable in the current state (e.g., trying to Open an already open container). We log the pre- and post-execution observations and convert them into real execution feedback $\mathcal{F}_{\text{real}}$, which captures observed state changes, failed manipulations, and missing affordances.

For *ALFRED* tasks, we execute the plan using the ALFWorld interface, adopting its discrete action format (e.g., Goto, TakeObject, OpenObject, MoveObject). The PDDL domain is constructed such that each high-level PDDL action has a one-to-one mapping to an *ALFWorld* action. During execution, *ALFWorld* returns success or failure signals together with updated observations.

Overall, the real execution stage provides grounded feedback that complements symbolic validation: even when a plan passes the PDDL validator, execution failures in the simulator reveal mismatches between the symbolic world and the embodied environment.

D. Self-Adaptive Symbolic Memory Details

D.1. When SASMem Is Queried

SASMem is queried *only when* SESIm produces a structured failure signal, i.e., either (i) symbolic validation fails,

yielding $\mathcal{F}_{\text{symbol}}$, or (ii) symbolic validation passes but real execution fails (or reveals unexpected postconditions), yielding $\mathcal{F}_{\text{real}}$. Intuitively, SASMem is not used as a generic prompt augmentation; instead, it is activated by *localized* failure evidence so that the returned information is directly actionable for plan/world refinement.

Query construction from structured feedback. Given a symbolic validation failure, SESIm extracts a compact query key from the failing action and the validator-reported error:

$$q_{\text{symbol}} = \langle \text{fail_action}, \text{error_type}, \text{error_content} \rangle.$$

Given a real execution failure or mismatch, SESIm extracts an execution query key:

$$q_{\text{real}} = \langle \text{fail_content} \rangle$$

where *fail_content* summarizes the grounded failure evidence near the failure point (e.g., affordance mismatch, unexpected state change, or interaction refusal). SESIm forwards these keys to a dedicated *MemoryManager*, which retrieves a small set of predicate-level *patches/hints* and returns them as $\mathcal{F}_{\text{SASMem}}$ for the next refinement call.

D.2. What Is Returned

SASMem maintains two complementary stores: *action memory* and *world memory*. The *MemoryManager* returns only a minimal set of high-density hints rather than raw trajectories.

Action reasoning memory. Action memory is indexed by symbolic failure signatures and is retrieved primarily using q_{symbol} . For the most compatible entries (matched by the failing action and error type, and optionally refined by error content), each entry provides: (i) a short list of *failure-associated predicates* (the top predicates most frequently implicated by this failure signature), which serves as a hint for which symbolic facts are likely missing or inconsistent in the current world model; and (ii) a single *successful prerequisite action pattern* (top-1), describing the most common local prerequisite step that precedes this failing action in successful plans, which guides plan repair by suggesting which prerequisite should be ensured or inserted before the failing action.

World modeling memory. World memory is indexed by grounded mismatch evidence and is retrieved using q_{real} jointly with the local failing action context. Each retrieved entry is predicate-aligned and provides: (i) a small set of predicate names with lightweight statistics indicating their tendency to be *missing* versus *mis-specified* under real interaction evidence (e.g., counts that accumulate missing-evidence vs error-evidence), which biases the VLM toward either adding a missing predicate instance or revising an incorrect one; and (ii) a single *probing action suggestion* (top-1) that most frequently surfaces interaction evidence for the predicate, enabling targeted exploration to confirm or refute the suspected symbolic gap.

D.3. Write-gating with Credit Assignment

SASMem updates are governed by a write-gating mechanism that performs *credit assignment* based on whether an injected hint actually improves the subsequent SESim outcome. Concretely, suppose a SESim iteration first fails at step t with

$$\langle \text{fail_step} = t, \text{fail_action} = a_t, \text{error_type} = e \rangle,$$

and the MemoryManager injects an action-memory entry associated with (a_t, e) . After refinement, SESim re-validates/re-executes and compares the next outcome against the original failure signature. The injected entry is treated as *effective* and its support statistic is increased if either: (i) the same error type e no longer occurs at the same failing action a_t (i.e., the original failure pattern is resolved), or (ii) the first failure moves to a strictly later step $t' > t$ (i.e., progress is made even if the task is not fully solved yet). Otherwise, if SESim still fails on the same action with the same symbolic error type, the entry is down-weighted via a symbolic-failure.

E. PDDL Domain File

For completeness and reproducibility, we include the full PDDL domain definitions used in our experiments for both

VirtualHome and *ALFRED*. These domains specify the types, predicates, and actions used for embodied planning in each environment.

E.1. VirtualHome Domain

```

1 (define (domain virtualHome)
2   (:requirements :adl :strips :typing )
3
4   (:types
5     agent
6     room
7     object
8   )
9
10  (:predicates
11    (sitting_state ?a - agent); ?a is
      seated.
12    (lying_state ?a - agent); ?a is
      lying down.
13    (sleeping_state ?a - agent) ; ?a is
      asleep.
14    (clean_state ?obj - object) ; ?obj
      is clean
15    (turnon_state ?obj - object) ; ?obj
      is turned on.
16    (open_state ?obj - object) ; ?obj is
      open
17
18    (grabbable ?obj - object) ; ?obj can
      be picked up.
19    (sittable ?obj - object) ; ?obj can
      be sat on.
20    (eatable ?obj - object) ; ?obj can
      be eaten.
21    (readable ?obj - object) ; ?obj can
      be read.
22    (drinkable ?obj - object) ; ?obj can
      be drunk.
23    (switchable ?obj - object) ; ?obj
      can be switched on/off.
24    (openable ?obj - object) ; ?obj can
      be opened.
25
26    (agentAtRoom ?a - agent ?r - room) ;
      ?a is in room ?r
27    (agentAtObject ?a - agent ?obj -
      object); ?a is at ?obj
28    (holdsRight ?a - agent ?obj - object
      ) ; ?a holds ?obj in right hand
29    (holdsLeft ?a - agent ?obj - object)
      ; ?a holds ?obj in left hand
30    (isHoldingLeft ?a - agent) ; ?a
      holds an object in left hand
31    (isHoldingRight ?a - agent) ; ?a
      holds an object in right hand
32    (onTop ?obj1 - object ?obj2 - object
      ) ; ?obj1 is on top of ?obj2
33    (insideObject ?obj1 - object ?obj2 -
      object); ?obj1 is inside ?obj2
34    (objAtRoom ?obj - object ?r - room);
      ?obj is in room ?r
35  )
36
```

```

37 (:action walkToRoom
38   :parameters (?a - agent ?from - room
39               ?to - room)
39   :precondition (and
40     (not(sitting_state ?a))
41     (agentAtRoom ?a ?from))
42   :effect (and
43     (agentAtRoom ?a ?to)
44     (not(agentAtRoom ?a ?from)))
45 )
46
47 (:action walkToObject
48   :parameters (?a - agent ?obj -
49               object ?r - room)
49   :precondition (and
50     (not (sitting_state ?a))
51     (objAtRoom ?obj ?r)
52     (agentAtRoom ?a ?r))
53   :effect (agentAtObject ?a ?obj)
54 )
55
56 (:action runToRoom
57   :parameters (?a - agent ?from - room
58               ?to - room)
58   :precondition (and
59     (not(sitting_state ?a))
60     (agentAtRoom ?a ?from))
61   :effect (and
62     (agentAtRoom ?a ?to)
63     (not(agentAtRoom ?a ?from)))
64 )
65
66 (:action runToObject
67   :parameters (?a - agent ?obj -
68               object ?r - room)
68   :precondition (and
69     (not (sitting_state ?a))
70     (objAtRoom ?obj ?r)
71     (agentAtRoom ?a ?r))
72   :effect (agentAtObject ?a ?obj)
73 )
74
75 (:action sit
76   :parameters (?a - agent ?obj -
77               object)
77   :precondition (and
78     (not (sitting_state ?a))
79     (agentAtObject ?a ?obj)
80     (sittable ?obj)
81   )
82   :effect (sitting_state ?a)
83 )
84
85 (:action standUp
86   :parameters (?a - agent)
87   :precondition (sitting_state ?a)
88   :effect (not (sitting_state ?a))
89 )
90
91 (:action grab
92   :parameters (?a - agent ?obj -
93               object)
93   :precondition (and
94     (grabbable ?obj)
95     (agentAtObject ?a ?obj)
96     (or (not ( isHoldingLeft ?a)) (not (
97         isHoldingRight ?a)))
98   :effect (and
99     (when (not(isHoldingRight ?a))
100       (and(holdsRight ?a ?obj) (
101         isHoldingRight ?a)))
101     (when (and(isHoldingRight ?a) (not(
102       isHoldingLeft ?a)))
103       (and(holdsLeft ?a ?obj) (
104         isHoldingLeft ?a)))
105   )
106 (:action agent_open_object
107   :parameters (?a - agent ?obj -
108               object)
108   :precondition (and
109     (openable ?obj)
110     (not(open_state ?obj))
111     (agentAtObject ?a ?obj)
112     (or (not (isHoldingRight ?a)) (not (
113       isHoldingLeft ?a)))
114   :effect (open_state ?obj)
115 )
116
117 (:action agent_close_object
118   :parameters (?a - agent ?obj -
119               object)
119   :precondition (and
120     (openable ?obj)
121     (open_state ?obj)
122     (agentAtObject ?a ?obj)
123     (or (not (isHoldingRight ?a)) (not (
124       isHoldingLeft ?a)))
125   :effect (not(open_state ?obj))
126 )
127
128 (:action putOn
129   :parameters (?a - agent ?obj1 -
130               object ?obj2 - object)
130   :precondition (and
131     (or (holdsRight ?a ?obj1) (holdsLeft
132       ?a ?obj1))
133     (agentAtObject ?a ?obj2)
134   )
135   :effect (and
136     (when(holdsRight ?a ?obj1)
137       (and(not(holdsRight ?a ?obj1))(
138         not(isHoldingRight ?a))))
139     (when(not(holdsRight ?a ?obj1))
140       (and(not(holdsLeft ?a ?obj1))(
141         not( isHoldingLeft ?a))))
142     (onTop ?obj1 ?obj2)
143   )
144 )
145 (:action putIn
146   :parameters (?a - agent ?obj1 -
147               object ?obj2 - object)
147   :precondition (and
148     (or (holdsRight ?a ?obj1) (holdsLeft
149       ?a ?obj1))

```

```

147 (agentAtObject ?a ?obj2)
148 (open_state ?obj2)
149 )
150 :effect (and
151 (when(holdsRight ?a ?obj1)
152 (and(not(holdsRight ?a ?obj1)) (
153 not(isHoldingRight ?a))))
154 (when(not(holdsRight ?a ?obj1))
155 (and(not(holdsLeft ?a ?obj1)) (
156 not(isHoldingLeft ?a))))
157 (insideObject ?obj1 ?obj2)
158 )
159 (:action switchOn
160 :parameters (?a - agent ?obj -
161 object)
162 :precondition (and
163 (switchable ?obj)
164 (not(turnon_state ?obj))
165 (agentAtObject ?a ?obj)
166 )
167 :effect (turnon_state ?obj)
168 )
169 (:action switchOff
170 :parameters (?a - agent ?obj -
171 object)
172 :precondition (and
173 (switchable ?obj)
174 (turnon_state ?obj)
175 (agentAtObject ?a ?obj)
176 )
177 :effect (not(turnon_state ?obj))
178 )
179 (:action drink
180 :parameters (?a - agent ?obj -
181 object)
182 :precondition (and
183 (drinkable ?obj)
184 (agentAtObject ?a ?obj)
185 )
186 :effect (and)
187 )
188 (:action touch
189 :parameters (?a - agent ?obj -
190 object)
191 :precondition (agentAtObject ?a ?obj
192 )
193 :effect (and)
194 )
195 (:action lookAt
196 :parameters (?a - agent ?obj -
197 object)
198 :precondition (agentAtObject ?a ?obj
199 )
200 :effect (and)
201 )
202 (:action wipe
203 :parameters (?a - agent ?obj -
204 object)

```

```

202 :precondition (and
203 (or (holdsRight ?a ?obj) (holdsLeft
204 ?a ?obj))
205 (agentAtObject ?a ?obj)
206 )
207 :effect (clean_state ?obj)
208 )
209 (:action wash
210 :parameters (?a - agent ?obj -
211 object)
212 :precondition (and
213 (or (holdsRight ?a ?obj) (holdsLeft
214 ?a ?obj))
215 (agentAtObject ?a ?obj)
216 )
217 :effect (clean_state ?obj)
218 )
219 (:action read
220 :parameters (?a - agent ?obj -
221 object)
222 :precondition (and
223 (or (holdsRight ?a ?obj) (holdsLeft
224 ?a ?obj))
225 (agentAtObject ?a ?obj)
226 (readable ?obj)
227 )
228 :effect (and)
229 )
230 (:action eat
231 :parameters (?a - agent ?obj -
232 object)
233 :precondition (and
234 (or (holdsRight ?a ?obj) (holdsLeft
235 ?a ?obj))
236 (agentAtObject ?a ?obj)
237 (eatable ?obj)
238 )
239 :effect (and)
240 )
241 (:action sleep
242 :parameters (?a - agent)
243 :precondition (or (sitting_state ?a)
244 (lying_state ?a))
245 :effect (sleeping_state ?a)
246 )
247 (:action wakeup
248 :parameters (?a - agent)
249 :precondition (sleeping_state ?a)
250 :effect (not(sleeping_state ?a))
251 )

```

E.2. ALFRED Domain

```

1 (define (domain alfred)
2 (:requirements
3 :adl
4 :typing
5 )

```

```

6 (:types
7   agent
8   receptacle
9   object
10 )
11
12 (:predicates
13   (agentAtReceptacle ?a - agent ?r -
14     receptacle) ; true if
15     the agent is at the receptacle
16   (objectAtReceptacle ?o - object ?r -
17     receptacle) ; true
18     if the object is at the
19     receptacle
20   (openable ?r - receptacle)
21   ;
22   true if a receptacle is openable
23   (cleanable ?o - object)
24   ;
25   true if the object can be
26   placed in a sink
27   (heatable ?o - object)
28   ;
29   true if the object can be
30   heated up in a microwave
31   (coolable ?o - object)
32   ;
33   true if the object can be
34   cooled in the fridge
35   (pickupable ?o - object)
36   ;
37   true if the object can be
38   picked up
39   (toggleable ?o - object)
40   ;
41   true if the object can be
42   turned on/off
43   (cleanTool ?r - receptacle)
44   (heatTool ?r - receptacle)
45   (coolTool ?r - receptacle)
46   (holds ?a - agent ?o - object)
47   ;
48   object ?o is held by agent ?a
49   (holdsAny ?a - agent)
50   ;
51   agent ?a holds an object
52   (opened ?r - receptacle)
53   ;
54   true if a receptacle is opened
55   (isClean ?o - object)
56   ;
57   true if the object has been
58   clean in sink
59   (isHot ?o - object)
60   ;
61   true if the object has been
62   heated up
63   (isCool ?o - object)
64   ;
65   true if the object has been
66   cooled
67   (isOn ?o - object)

```

```

33   ; true if the object is on
34 )
35 (:action GotoReceptacle
36   :parameters (?a - agent ?from_recep
37     - receptacle ?to_recep -
38     receptacle)
39   :precondition (and
40     (agentAtReceptacle ?a ?
41       from_recep)
42     (not (agentAtReceptacle ?a ?
43       to_recep)))
44   )
45   :effect (and
46     (not (agentAtReceptacle ?a ?
47       from_recep))
48     (agentAtReceptacle ?a ?to_recep)
49   )
50 )
51 (:action OpenReceptacle
52   :parameters (?a - agent ?r -
53     receptacle)
54   :precondition (and
55     (openable ?r)
56     (agentAtReceptacle ?a ?r)
57     (not (opened ?r))
58   )
59   :effect (and
60     (opened ?r)
61   )
62 )
63 (:action CloseObject
64   :parameters (?a - agent ?r -
65     receptacle)
66   :precondition (and
67     (openable ?r)
68     (agentAtReceptacle ?a ?r)
69     (opened ?r)
70   )
71   :effect (and
72     (not (opened ?r))
73   )
74 )
75 (:action TakeObject
76   :parameters (?a - agent ?o - object
77     ?r - receptacle)
78   :precondition
79     (and
80       (pickupable ?o)
81       (agentAtReceptacle ?a ?r)
82       (objectAtReceptacle ?o ?r)
83       (not (holdsAny ?a)) ; agent
84       's hands are empty.
85       (or (not (openable ?r)) (
86         opened ?r))
87     )
88   :effect
89     (and
90       (not (objectAtReceptacle ?o
91         ?r))
92       (holds ?a ?o)
93       (holdsAny ?a)
94     )
95 )

```

```

86
87 (:action MoveObject
88   :parameters (?a - agent ?o - object
89               ?r - receptacle )
89   :precondition (and
90                 (holds ?a ?o)
91                 (agentAtReceptacle ?a ?r)
92                 (or (not (openable ?r)) (
93                     opened ?r))
94                 )
94   :effect (and
95           (objectAtReceptacle ?o ?r)
96           (not (holds ?a ?o))
97           (not (holdsAny ?a))
98           )
99 )
100
101 (:action CleanObject
102   :parameters (?a - agent ?o - object
103               ?r - receptacle)
103   :precondition (and
104                 (cleanable ?o)
105                 (cleanTool ?r)
106                 (agentAtReceptacle ?a ?r)
107                 (holds ?a ?o)
108                 )
109   :effect (and
110           (isClean ?o)
111           )
112 )
113
114 (:action HeatObject
115   :parameters (?a - agent ?o - object
116               ?r - receptacle)
116   :precondition (and
117                 (heatable ?o)
118                 (heatTool ?r)
119                 (agentAtReceptacle ?a ?r)
120                 (holds ?a ?o)
121                 )
122   :effect (and
123           (isHot ?o)
124           (not (isCool ?o))
125           )
126 )
127
128 (:action CoolObject
129   :parameters (?a - agent ?o - object
130               ?r - receptacle )
130   :precondition (and
131                 (coolable ?o)
132                 (coolTool ?r)
133                 (agentAtReceptacle ?a ?r)
134                 (holds ?a ?o)
135                 )
136   :effect (and
137           (isCool ?o)
138           (not (isHot ?o))
139           )
140 )
141
142 (:action ToggleObject
143   :parameters (?a - agent ?o - object
144               ?r - receptacle)
144   :precondition (and

```

```

145         (toggleable ?o)
146         (agentAtReceptacle ?a ?r)
147         (objectAtReceptacle ?o ?r)
148         )
149   :effect (and
150           (when (isOn ?o)
151               (not (isOn ?o)))
152           (when (not (isOn ?o))
153               (isOn ?o))
154           )
155 )
156
157 )

```